Animations in Blender

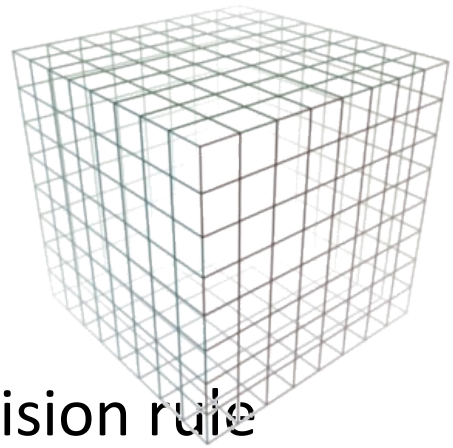
- Keyframes
- Frames
- Camera animation; track an object
- Animations in Python

Creating new meshes in blender

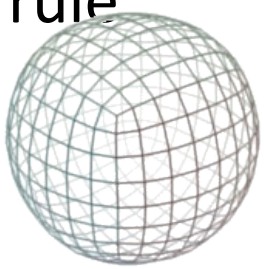
```python
import bpy

def frame_change_pre(scene):
    # A triangle that shifts in the z direction
    zshift = scene.frame_current * 0.02

    vertices = [(-1, -1, zshift), (1, -1, zshift), (0, 1, zshift)]
    triangles = [(0, 1, 2)]
    object = bpy.data.objects["Example"]
    object.data.clear_geometry()
    object.data.from_pydata(vertices, [], triangles)

me = bpy.data.meshes.new('ExampleMesh')
ob = bpy.data.objects.new('Example', me)
bpy.context.scene.collection.objects.link(ob)

bpy.app.handlers.frame_change_pre.append(frame_change_pre)
```

Prevent running conditions when modifying Blenders data:

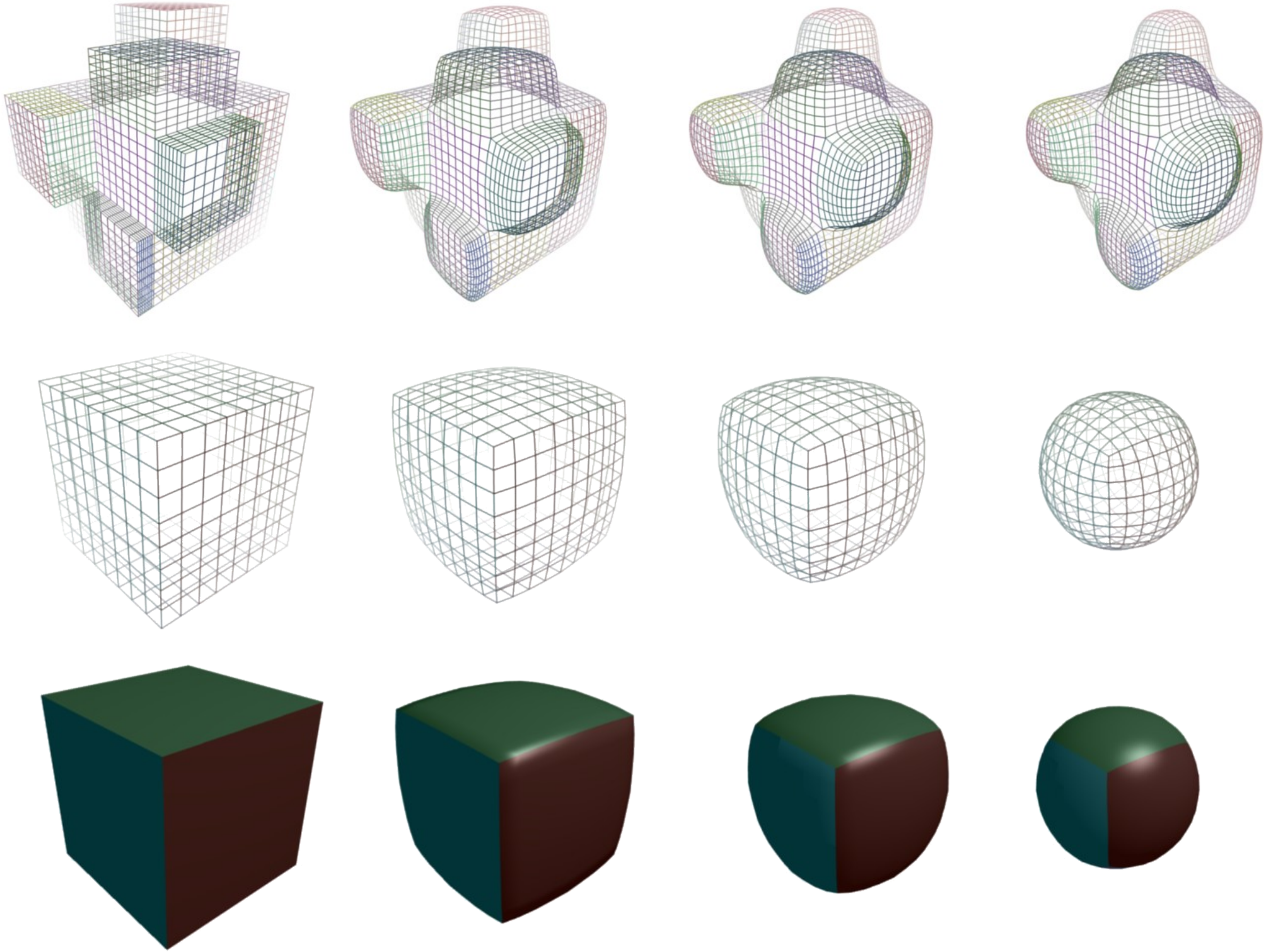
```
def playback_start(scene):
    scene.render.use_lock_interface = True

def playback_end(scene):
    scene.render.use_lock_interface = False

bpy.app.handlers.animation_playback_pre.append(playback_start)
bpy.app.handlers.animation_playback_post.append(playback_end)
```



- Face vertices:
  - P(t) = vertex given by the Catmull-Clark subdivision rule
- Edge vertices:
  - P_0 = midpoint of the edge
  - P_1 = vertex given by the Catmull-Clark subdivision rule
  - $P(t) = (1-t)P_0 + tP_1$.

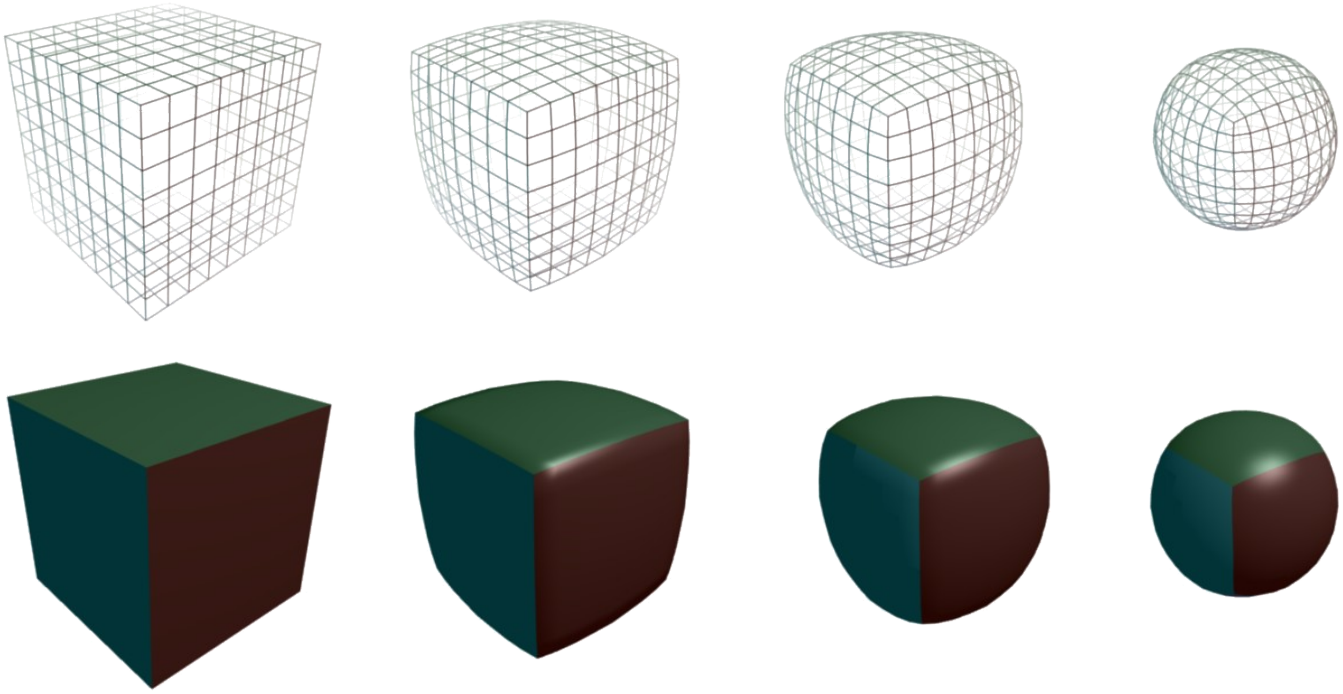


- Vertex vertices (old ones):
  - P_0 = original vertex
  - P_1 = vertex given by the Catmull-Clark subdivision rule
  - $P(t) = (1-t)P_0 + tP_1$.



Animations

- Generate a smooth animation showing how the geometry deforms after **n** Catmull Clark iterations:



Animations (cont.)

Get the current frame number to compute parameter **t** (time)

```
strt = scn.frame_start
end  = scn.frame_end
num  = end -strt
curr = scn.frame_current
time = float(curr - strt)/num
#time in [0,1]
```

Option 1

- Call catmullClark(mesh, n, t) at each frame.
- Since the input mesh has to be always the same (the active mesh), <u>the input mesh needs to be preserved</u>: the subdivision has to be applied to a copy of the object.
- The copy must be deleted at the beginning of the next frame.

Option 2

- First frame: generate the Catmull-Clark subdivision for **time** = 0.0 and **time**= 1.0, storing the vextex coordinates of both. Display the first one.
- Rest of frames:modify the geometry of the active mesh, according to the value of parameter **time** interpolating the pre-computed coordinates.
- Altering data here while rendering requires care. See previous slide about avoiding running conditions.

Useful methods and idioms

```
# you may change the name of an object:
ob.name = “NewNameIChoose”

# you can hide the object:
ob.hide_viewport = True

# There is an operator to delete the selected
# objects:
bpy.ops.object.delete()

# You can de_select the active object:
bpy.context.active_object.select_set(False)
# or select another:
ob.select_set(True)
```

Useful methods and idioms (II)

```
# Then select and erase those with “thing”
# in their name:
Mshs = []

for itm in bpy.data.objects:
    if "thing" in itm.name:
        mshs.append(itm.data)  # save for later deleting
        itm.select_set(True)

bpy.ops.object.delete()        #delete all selected
# and now delete all the orfan meshes:
for msh in mshs:
    bpy.data.meshes.remove(msh)
```

```
# You may duplicate the active object:
bpy.ops.object.duplicate_move()
```